

# Отчёт по лабораторной работе №7

## Компьютерный практикум по статистическому анализу данных

### Введение в работу с данными

Выполнил: Исаев Булат Абубакарович,  
НПИбд-01-22, 1132227131

### Содержание

|      |                                                                                                                   |    |
|------|-------------------------------------------------------------------------------------------------------------------|----|
| 1    | Цель работы .....                                                                                                 | 1  |
| 2    | Выполнение лабораторной работы.....                                                                               | 2  |
| 2.1  | Julia для науки о данных .....                                                                                    | 2  |
| 2.2  | Считывание данных.....                                                                                            | 2  |
| 2.3  | Запись данных в файл .....                                                                                        | 4  |
| 2.4  | Словари .....                                                                                                     | 5  |
| 2.5  | DataFrames.....                                                                                                   | 6  |
| 2.6  | RDatasets.....                                                                                                    | 6  |
| 2.7  | Работа с переменными отсутствующего типа (Missing Values) .....                                                   | 7  |
| 2.8  | FileIO .....                                                                                                      | 8  |
| 2.9  | Обработка данных: стандартные алгоритмы машинного обучения в Julia.<br>Кластеризация данных. Метод k-средних..... | 9  |
| 2.10 | Кластеризация данных. Метод k ближайших соседей .....                                                             | 14 |
| 2.11 | Обработка данных. Метод главных компонент.....                                                                    | 16 |
| 2.12 | Обработка данных. Линейная регрессия .....                                                                        | 16 |
| 2.13 | Самостоятельное выполнение .....                                                                                  | 19 |
| 3    | Вывод.....                                                                                                        | 23 |
| 4    | Список литературы. Библиография .....                                                                             | 23 |

## 1 Цель работы

Основной целью работы является изучение специализированных пакетов Julia для обработки данных.

## 2 Выполнение лабораторной работы

### 2.1 Julia для науки о данных

В Julia для обработки данных используются наработки из других языков программирования, в частности, из R и Python.

### 2.2 Считывание данных

Перед тем, как начать проводить какие-либо операции над данными, необходимо их откуда-то считать и возможно сохранить в определённой структуре.

Довольно часто данные для обработки содержатся в csv-файле, имеющим текстовый формат, в котором данные в строке разделены, например, запятыми, и соответствуют ячейкам таблицы, а строки данных соответствуют строкам таблицы. Также данные могут быть представлены в виде фреймов или множеств.

В Julia для работы с такого рода структурами данных используют пакеты CSV, DataFrames, RDatasets, FileIO (рис. 1):

```
[*]: # Обновление окружения:
using Pkg
Pkg.update
# Установка пакетов:
using Pkg
for p in ["CSV", "DataFrames", "RDatasets", "FileIO"]
    Pkg.add(p)
end
using CSV, DataFrames, DelimitedFiles
```

```
Resolving package versions...
Installed WorkerUtilities — v1.6.1
Installed DataValueInterfaces — v1.0.0
Installed SentinelArrays — v1.4.8
Installed WeakRefStrings — v1.4.2
Installed InlineStrings — v1.4.5
Installed TableTraits — v1.0.1
Installed PooledArrays — v1.4.3
Installed Tables — v1.12.1
Installed FilePathsBase — v0.9.24
Installed CSV — v0.10.15
```

Рис. 1: Установка пакетов

Предположим, что у нас в рабочем каталоге с проектом есть файл с данными programminglanguages.csv, содержащий перечень языков программирования и год их создания. Тогда для заполнения массива данными для последующей обработки требуется считать данные из исходного файла и записать их в соответствующую структуру (рис. 2):

```
[10]: # Считывание данных и их запись в структуру:
P = CSV.File("programminglanguages.csv") |> DataFrame

# Функция определения по названию языка программирования года его создания:
function language_created_year(P, language::String)
    loc = findfirst(P[:,2].==language)
    return P[loc,1]
end

# Пример вызова функции и определение даты создания языка Python:
language_created_year(P, "Python")

[10]: 1991
```

Рис. 2: Считывание данных и запись в структуру

Пример для Julia (рис. 3):

```
[11]: # Пример вызова функции и определение даты создания языка Julia:
language_created_year(P, "Julia")

[11]: 2012
```

Рис. 3: Пример

В следующем примере при вызове функции, в качестве аргумента которой указано слово *julia*, написанное со строчной буквы (рис. 4):

```
[12]: # Пример вызова функции и определение даты создания языка julia:
language_created_year_v2(P, "julia")

[12]: 2012
```

Рис. 4: Поиск “julia” со строчной буквы

Для того, чтобы убрать в функции зависимость данных от регистра, необходимо изменить исходную функцию следующим образом (рис. 5):

```
[13]: # Функция определения по названию языка программирования
# года его создания (без учёта регистра):
function language_created_year_v2(P, language::String)
    loc = findfirst(lowercase.(P[:,2]).==lowercase.(language))
    return P[loc,1]
end

# Пример вызова функции и определение даты создания языка julia:
language_created_year_v2(P, "julia")

[13]: 2012
```

Рис. 5: Изменение исходной функции

Можно считывать данные построчно, с элементами, разделенными заданным разделителем (рис. 6):

```
[14]: # Построчное считывание данных с указанием разделителя:  
Tx = readdlm("programminglanguages.csv", ',')
```

```
[14]: 22x2 Matrix{Any}:  
      "year"  "language"  
1957      "Fortran"  
1958      "Lisp"  
1959      "COBOL"  
1964      "BASIC"  
1970      "Pascal"  
1972      "C"  
1978      "SQL"  
1980      "C++"  
1983      "Objective-C"  
1987      "Perl"  
1990      "Haskell"  
1991      "Python"  
1993      "Lua"  
1995      "Java"  
1995      "JavaScript"  
1995      "PHP"  
2000      "C#"  
2003      "Groovy"  
2003      "Scala"  
2012      "Julia"  
2014      "Swift"
```

Рис. 6: Построчное считывание данных

## 2.3 Запись данных в файл

Предположим, что требуется записать имеющиеся данные в файл. Для записи данных в формате CSV можно воспользоваться следующим вызовом (рис. 7):

```
[15]: # Запись данных в CSV-файл:  
CSV.write("programming_languages_data2.csv", P)  
  
[15]: "programming_languages_data2.csv"
```

Рис. 7: Запись данных в файл

Можно задать тип файла и разделитель данных (рис. 8):

```
[16]: # Пример записи данных в текстовый файл с разделителем ',':  
writedlm("programming_languages_data.txt", Tx, ',')  
  
[17]: # Пример записи данных в текстовый файл с разделителем '-':  
writedlm("programming_languages_data2.txt", Tx, '-')
```

Рис. 8: Пример с указанием типа данных и разделителем данных

Можно проверить, используя `readdlm`, корректность считывания созданного текстового файла (рис. 9):

```
[18]: # Построчное считывание данных с указанием разделителя:
P_new_delim = readdlm("programming_languages_data2.txt", '-')

[18]: 22×2 Matrix{Any}:
      "year"  "language"
1957      "Fortran"
1958      "Lisp"
1959      "COBOL"
1964      "BASIC"
1970      "Pascal"
1972      "C"
1978      "SQL"
1980      "C++"
1983      "Objective-C"
1987      "Perl"
1990      "Haskell"
1991      "Python"
1993      "Lua"
1995      "Java"
1995      "JavaScript"
1995      "PHP"
2000      "C#"
2003      "Groovy"
2003      "Scala"
2012      "Julia"
2014      "Swift"
```

Рис. 9: Проверка корректности считывания созданного текстового файла

## 2.4 Словари

При работе с данными бывает удобно записать их в формате словаря.

Предположим, что словарь должен содержать перечень всех языков программирования и года их создания, при этом при указании года выводить все языки программирования, созданные в этом году.

При инициализации словаря можно задать конкретные типы данных для ключей и значений (рис. 10):

```
[19]: # Инициализация словаря:
dict = Dict{Integer,Vector{String}}{()}

[19]: Dict{Integer, Vector{String}}{()}
```

Рис. 10: Инициализация словаря

Можно инициировать пустой словарь, не задавая строго структуру (рис. 11):

```
[20]: # Инициализация словаря:
dict2 = Dict{Any, Any}()

[20]: Dict{Any, Any}()
```

Рис. 11: Инициализация пустого словаря

Далее требуется заполнить словарь ключами и годами, которые содержат все языки программирования, созданные в каждом году, в качестве значений (рис. 12):

```
[21]: # Заполнение словаря данными:
      for i = 1:size(P,1)
          year,lang = P[i,:];
          if year in keys(dict)
              dict[year] = push!(dict[year],lang)
          else
              dict[year] = [lang]
          end
      end
```

Рис. 12: Заполнение словаря данными

В результате при вызове словаря можно, выбрав любой год, узнать, какие языки программирования были созданы в этом году (рис. 13):

```
[22]: # Пример определения в словаре языков программирования, созданных в 2003 год
      dict[2003]

[22]: 2-element Vector{String}:
       "Groovy"
       "Scala"
```

Рис. 13: Пример работы словаря

## 2.5 DataFrames

Работа с данными, записанными в структуре DataFrame, позволяет использовать индексацию и получить доступ к столбцам по заданному имени заголовка или по индексу столбца.

На примере с данными о языках программирования и годах их создания зададим структуру DataFrame (рис. 14):

```
[24]: # Подгружаем пакет DataFrames:
      using DataFrames

[25]: # Задаём переменную со структурой DataFrame:
      df = DataFrame(year = P[:,1], language = P[:,2])
      # Вывод всех значения столбца year:
      df[!,:year]
      # Получение статистических сведений о фрейме:
      describe(df)
```

```
[25]: 2×7 DataFrame
```

| Row | variable | mean     | min   | median   | max   | nmissing | eltype   |
|-----|----------|----------|-------|----------|-------|----------|----------|
|     | Symbol   | Union... | Any   | Union... | Any   | Int64    | DataType |
| 1   | year     | 1985.67  | 1957  | 1990.0   | 2014  | 0        | Int64    |
| 2   | language |          | BASIC |          | Swift | 0        | String15 |

Рис. 14: Пример создания структуры DataFrame

## 2.6 RDatasets

С данными можно работать также как с наборами данных через пакет RDatasets языка R (рис. 15):

```
[28]: # Подгружаем пакет RDatasets:
      using RDatasets

[29]: # Задаём структуру данных в виде набора данных:
      iris = dataset("datasets", "iris")
      # Определения типа переменной:
      typeof(iris)

[29]: DataFrame
```

Рис. 15: Работа с пакетом RDatasets

Пакет RDatasets также предоставляет возможность с помощью `describe` получить основные статистические сведения о каждом столбце в наборе данных (рис. 16):

```
[30]: describe(iris)
```

[30]: 5×7 DataFrame

| Row | variable    | mean     | min    | median   | max       | nmissing | eltype                          |
|-----|-------------|----------|--------|----------|-----------|----------|---------------------------------|
|     | Symbol      | Union... | Any    | Union... | Any       | Int64    | DataType                        |
| 1   | SepalLength | 5.84333  | 4.3    | 5.8      | 7.9       | 0        | Float64                         |
| 2   | SepalWidth  | 3.05733  | 2.0    | 3.0      | 4.4       | 0        | Float64                         |
| 3   | PetalLength | 3.758    | 1.0    | 4.35     | 6.9       | 0        | Float64                         |
| 4   | PetalWidth  | 1.19933  | 0.1    | 1.3      | 2.5       | 0        | Float64                         |
| 5   | Species     |          | setosa |          | virginica | 0        | CategoricalValue{String, UInt8} |

Рис. 16: Получение основных статических сведений о каждом столбце в наборе данных

## 2.7 Работа с переменными отсутствующего типа (Missing Values)

Пакет DataFrames позволяет использовать так называемый «отсутствующий» тип (рис. 17):

```
[31]: # Отсутствующий тип:
      a = missing
      typeof(a)

[31]: Missing
```

Рис. 17: Использование «отсутствующего» типа

В операции сложения числа и переменной с отсутствующим типом значение также будет иметь отсутствующий тип (рис. 18):

```
[32]: # Пример операции с переменной отсутствующего типа:
      a + 1

[32]: missing
```

Рис. 18: Операция сложения числа и переменной с отсутствующим типом

Приведём пример работы с данными, среди которых есть данные с отсутствующим типом.

Предположим есть перечень продуктов, для которых заданы калории. В массиве значений калорий есть значение с отсутствующим типом (рис. 19):

```
[33]: # Определение перечня продуктов:
foods = ["apple", "cucumber", "tomato", "banana"]
# Определение калорий:
calories = [missing, 47, 22, 105]

# Определение типа переменной:
typeof(calories)

[33]: Vector{Union{Missing, Int64}} (alias for Array{Union{Missing, Int64}, 1})
```

Рис. 19: Пример работы с данными, среди которых есть данные с отсутствующим типом

При попытке получить среднее значение калорий, ничего не получится из-за наличия переменной с отсутствующим типом.

Для решения этой проблемы необходимо игнорировать отсутствующий тип (рис. 20):

```
[34]: # Подключаем пакет Statistics:
using Statistics

[35]: # Определение среднего значения:
mean(calories)

# Определение среднего значения без значений с отсутствующим типом:
mean(skipmissing(calories))

[35]: 58.0
```

Рис. 20: Игнорирование отсутствующего типа

Далее показано, как можно сформировать таблицы данных и объединить их в один фрейм (рис. 21):

```
[41]: # Замена missing значений средним
calories_filled = coalesce(DF.calories, mean(skipmissing(DF.calories)))
DF_filled = DataFrame(item = DF.item, calories = calories_filled, price = DF.price)
println("\nDataFrame с заполненными missing значениями:")
println(DF_filled)
```

DataFrame с заполненными missing значениями:  
4×3 DataFrame

| Row | item     | calories | price   |
|-----|----------|----------|---------|
|     | String   | Real     | Float64 |
| 1   | apple    | 58.0     | 0.85    |
| 2   | cucumber | 47       | 1.6     |
| 3   | tomato   | 22       | 0.8     |
| 4   | banana   | 105      | 0.6     |

Рис. 21: Формирование таблиц данных и их объединение в один фрейм

## 2.8 FileIO

В Julia можно работать с так называемыми «сырыми» данными, используя пакет FileIO.



Попробуем посмотреть, как Julia работает с изображениями.

Подключим соответствующий пакет (рис. 22):

```
•[*]: # Подключаем пакет FileIO:  
      using FileIO  
  
[*]: import Pkg  
      Pkg.add("ImageIO")
```

Рис. 22: Подключение пакетов

Загрузим изображение (в данном случае логотип Julia) (рис. 23):

```
[58]: # Загрузка изображения:  
      X1 = load("julia.png")  
      display(X1)
```



Рис. 23: Загрузка изображения

Julia хранит изображение в виде множества цветов (рис. 24):

```
[*]: # Определение типа и размера данных:  
      @show typeof(X1);  
      @show size(X1);
```

Рис. 24: Определение типа и размера данных

## 2.9 Обработка данных: стандартные алгоритмы машинного обучения в Julia. Кластеризация данных. Метод k-средних

Задача кластеризации данных заключается в формировании однородной группы упорядоченных по какому-то признаку данных.

Метод k-средних позволяет минимизировать суммарное квадратичное отклонение точек кластеров от центров этих кластеров.

Рассмотрим задачу кластеризации данных на примере данных о недвижимости. Файл с данными `houses.csv` содержит список транзакций с недвижимостью в районе Сакраменто, о которых было сообщено в течение определённого числа дней.

Сначала подключим необходимые для работы пакеты (рис. 25):

```
[*]: # Загрузка пакетов:
import Pkg
Pkg.add("DataFrames")
Pkg.add("Statistics")
using DataFrames
using CSV
import Pkg
Pkg.add("Plots")
```

Рис. 25: Подключение нужных пакетов

Затем загрузим данные (рис. 26):

```
[75]: using CSV, DataFrames, Downloads
using Statistics # для mean

# Скачиваем Sacramento.csv (аналог houses.csv из лабораторной)
url = "https://raw.githubusercontent.com/selva86/datasets/master/Sacramento.csv"
filename = "Sacramento.csv" # Имя файла как в оригинале

if !isfile(filename)
    Downloads.download(url, filename)
    println("Файл $filename успешно скачан! (985 строк о недвижимости в Сакраменто)")
else
    println("Файл $filename уже есть – всё ок!")
end

# Теперь читаем (заменяем houses на Sacramento в лабораторной)
houses = CSV.File(filename) |> DataFrame
println("Размер датасета: ", size(houses))
println("Первые 15 строк:")
println(first(houses, 15))
```

Файл Sacramento.csv уже есть – всё ок!  
 Размер датасета: (932, 9)  
 Первые 15 строк:  
 15×9 DataFrame

| Row | city<br>String15 | zip<br>String7 | beds<br>Int64 | baths<br>Float64 | sqft<br>Int64 | type<br>String15 | price<br>Int64 | latitude<br>Float64 | longitude<br>Float64 |
|-----|------------------|----------------|---------------|------------------|---------------|------------------|----------------|---------------------|----------------------|
| 1   | SACRAMENTO       | z95838         | 2             | 1.0              | 836           | Residential      | 59222          | 38.6319             | -121.435             |
| 2   | SACRAMENTO       | z95823         | 3             | 1.0              | 1167          | Residential      | 68212          | 38.4789             | -121.431             |
| 3   | SACRAMENTO       | z95815         | 2             | 1.0              | 796           | Residential      | 68880          | 38.6183             | -121.444             |
| 4   | SACRAMENTO       | z95815         | 2             | 1.0              | 852           | Residential      | 69307          | 38.6168             | -121.439             |
| 5   | SACRAMENTO       | z95824         | 2             | 1.0              | 797           | Residential      | 81900          | 38.5195             | -121.436             |
| 6   | SACRAMENTO       | z95841         | 3             | 1.0              | 1122          | Condo            | 89921          | 38.6626             | -121.328             |
| 7   | SACRAMENTO       | z95842         | 3             | 2.0              | 1104          | Residential      | 90895          | 38.6817             | -121.352             |
| 8   | SACRAMENTO       | z95820         | 3             | 1.0              | 1177          | Residential      | 91002          | 38.5351             | -121.481             |
| 9   | RANCHO_CORDOVA   | z95670         | 2             | 2.0              | 941           | Condo            | 94905          | 38.6212             | -121.271             |
| 10  | RIO_LINDA        | z95673         | 3             | 2.0              | 1146          | Residential      | 98937          | 38.7009             | -121.443             |
| 11  | SACRAMENTO       | z95838         | 3             | 2.0              | 909           | Residential      | 100309         | 38.6377             | -121.452             |
| 12  | SACRAMENTO       | z95823         | 3             | 2.0              | 1289          | Residential      | 106250         | 38.4707             | -121.459             |
| 13  | SACRAMENTO       | z95815         | 1             | 1.0              | 871           | Residential      | 106852         | 38.6187             | -121.436             |
| 14  | SACRAMENTO       | z95822         | 3             | 1.0              | 1020          | Residential      | 107502         | 38.4822             | -121.493             |
| 15  | SACRAMENTO       | z95842         | 2             | 2.0              | 1022          | Residential      | 108750         | 38.6729             | -121.359             |

Рис. 26: Загрузка данных

Построим график цен на недвижимость в зависимости от площади (рис. 27):

```
[85]: # Фильтрация данных по заданному условию:
filter_houses = houses[houses[:,sq_ft].>0,:]
# Построение графика:
x = filter_houses[:,sq_ft]
y = filter_houses[:,price]
scatter(x,y, markersize=3)
```

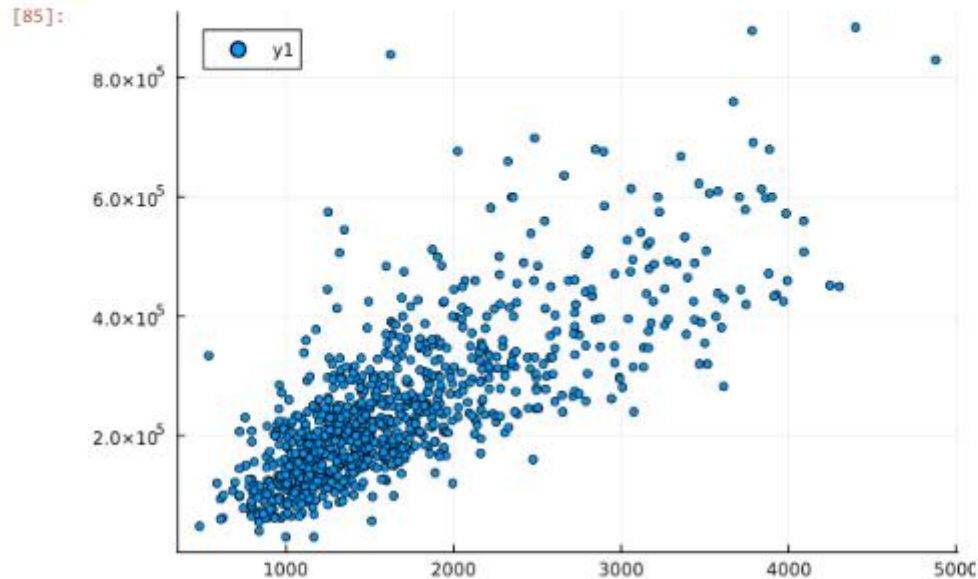


Рис. 27: Построение графика цен на недвижимость в зависимости от площади

Для того чтобы избавиться от “артефактов”, можно отфильтровать и исключить такие значения, получить более корректный график цен (рис. 28):

```
[86]: # Фильтрация данных по заданному условию:
filter_houses = houses[houses[! , :sq_ft].>0, :]
# Построение графика
x = filter_houses[! , :sq_ft]
y = filter_houses[! , :price]
scatter(x,y)
```

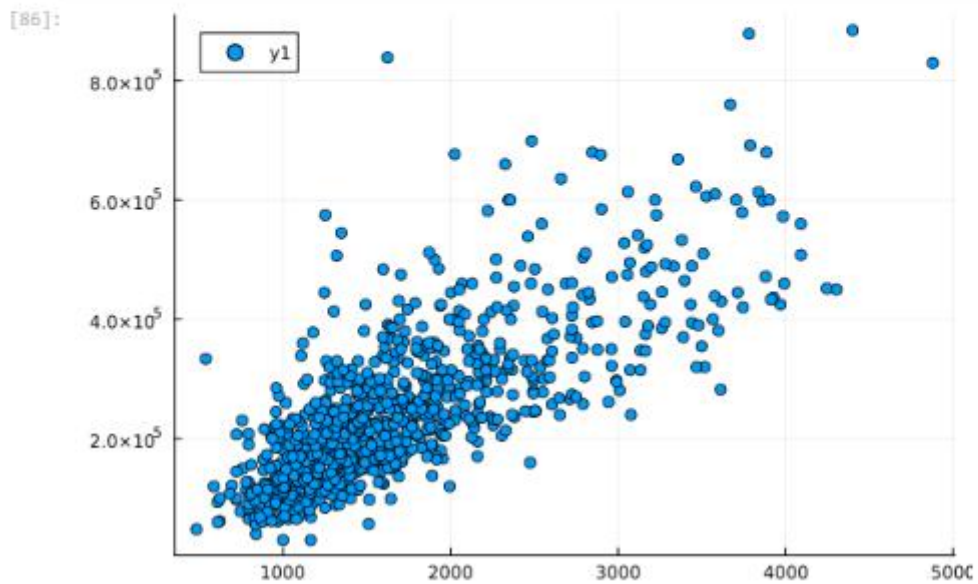


Рис. 28: Построение графика без “артефактов”

Построим график, обозначив каждый кластер отдельным цветом (рис. 29):

Средние цены для каждого типа домов:

4x2 DataFrame

| Row | type         | mean_price |
|-----|--------------|------------|
|     | String15     | Float64    |
| 1   | Residential  | 2.39186e5  |
| 2   | Condo        | 1.50082e5  |
| 3   | Multi-Family | 2.24535e5  |
| 4   | Unkown       | 275000.0   |

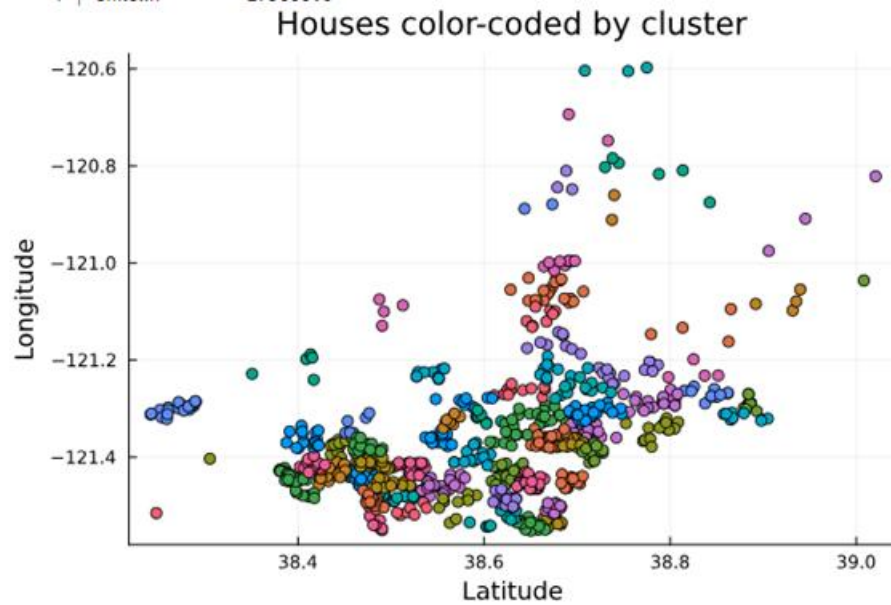


Рис. 29: Построение графика с кластерами разных цветов

Построим график, раскрасив кластеры по почтовому индексу (рис. 30):

```
[87]: unique_zips = unique(filter_houses[:, :zip])
      zips_figure = plot(legend = false)
      for uzip in unique_zips
          subs = filter_houses[filter_houses[:, :zip].==uzip, :]
          x = subs[:, :latitude]
          y = subs[:, :longitude]
          scatter!(zips_figure, x, y)
      end

      xlabel!("Latitude")
      ylabel!("Longitude")
      title!("Houses color-coded by zip code")
      display(zips_figure)
```



Рис. 30: Построение графика с кластерами разных цветов по почтовому индексу

## 2.10 Кластеризация данных. Метод k ближайших соседей

Отобразим на графике соседей выбранного объекта недвижимости (рис. 31):

```
[108]: # Подключение пакета NearestNeighbors:
import Pkg
Pkg.add("NearestNeighbors")
using NearestNeighbors

knearest = 10
id = 70
point = X[:,id]

# Поиск ближайших соседей:
kdtree = KDTree(X)
idxs, dists = knn(kdtree, point, knearest, true)

# Все объекты недвижимости:
x = filter_houses[:, :latitude];
y = filter_houses[:, :longitude];
scatter(x, y)

# Соседи:
x = filter_houses[idxs, :latitude];
y = filter_houses[idxs, :longitude];
scatter!(x, y)

Resolving package versions...
Project No packages added to or removed from `C:\Users\Пользователь\.julia\environments\v1.12\Project.toml`
Manifest No packages added to or removed from `C:\Users\Пользователь\.julia\environments\v1.12\Manifest.toml`
```

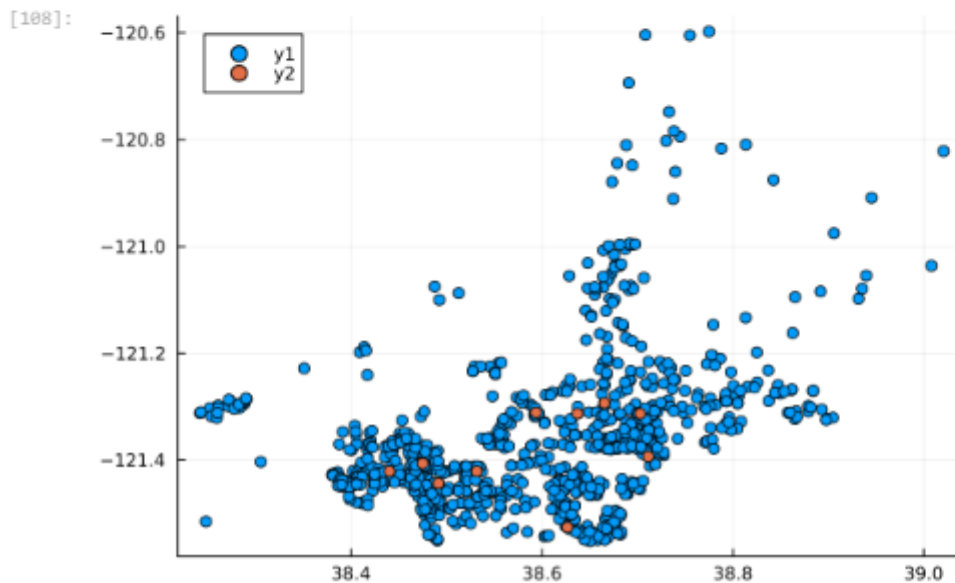


Рис. 31: Отображение на графике соседей выбранного объекта недвижимости

Используя индексы `idxs` и функцию `:city` для индексации в DataFrame `filter_houses`, можно определить районы соседних домов (рис. 32):

```
[110]: # Фильтрация по районам соседних домов:
cities = filter_houses[idxs, :city]

[110]: 10-element PooledArrays.PooledVector{String15, UInt32, Vector{UInt32}}:
"SACRAMENTO"
"SACRAMENTO"
"SACRAMENTO"
"RANCHO_CORDOVA"
"CITRUS_HEIGHTS"
"CITRUS_HEIGHTS"
"SACRAMENTO"
"ANTELOPE"
"CARMICHAEL"
"SACRAMENTO"
```

Рис. 32: Определение районов соседних домов

## 2.11 Обработка данных. Метод главных компонент

Метод главных компонент (Principal Components Analysis, PCA) позволяет уменьшить размерность данных, потеряв наименьшее количество полезной информации. Метод имеет широкое применение в различных областях знаний, например, при визуализации данных, компрессии изображений, в эконометрике, некоторых гуманитарных предметных областях, например, в социологии или в политологии.

На примере с данными о недвижимости попробуем уменьшить размеры данных о цене и площади из набора данных домов (рис. 33):

```
[114]: # Фрейм с указанием площади и цены недвижимости:
F = filter_houses[:, :sq_ft, :price]
# Конвертация данных в массив:
F = convert(Array{Float64,2}, F)

# Подключение пакета MultivariateStats:
import Pkg
Pkg.add("MultivariateStats")
using MultivariateStats

# Приведение типов данных к распределению для PCA:
M = fit(PCA, F)
# Выделение значений главных компонент в отдельную переменную:
Xr = reconstruct(M, y)
# Построение графика с выделением главных компонент:
scatter(F[1, :], F[2, :])
scatter!(Xr[1, :], Xr[2, :])
```

Рис. 33: Попытка уменьшения размера данных о цене и площади из набора данных домов

## 2.12 Обработка данных. Линейная регрессия

Регрессионный анализ представляет собой набор статистических методов исследования влияния одной или нескольких независимых переменных (регрессоров) на зависимую (критериальная) переменную. Терминология зависимых и независимых переменных отражает лишь математическую зависимость переменных, а не причинноследственные отношения.



Наиболее распространённый вид регрессионного анализа — линейная регрессия, когда находят линейную функцию, которая согласно определённым математическим критериям наиболее соответствует данным.

Зададим случайный набор данных (можно использовать и полученные экспериментальным путём какие-то данные). Попробуем найти для данных лучшее соответствие (рис. 34):

```
[118]: xvals = repeat(1:0.5:10,inner=2)
       yvals = 3 .+ xvals + 2*rand(length(xvals)) .- 1
       scatter(xvals,yvals,color=:black,leg=false)
```

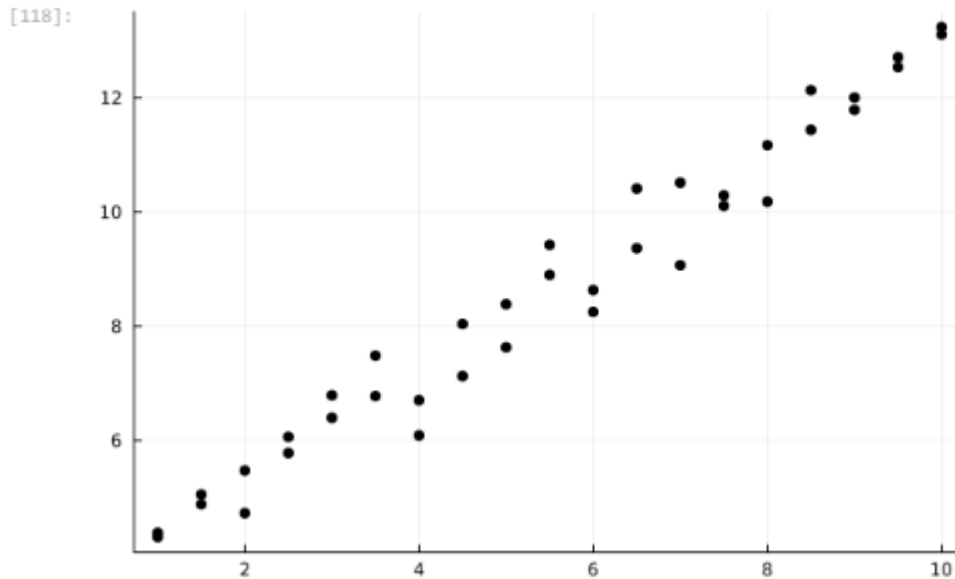


Рис. 34: Исходные данные

Определим функцию линейной регрессии. Применим функцию линейной регрессии для построения соответствующего графика значений (рис. 35):

```
[120]: function find_best_fit(xvals,yvals)
        meanx = mean(xvals)
        meany = mean(yvals)
        stdx = std(xvals)
        stdy = std(yvals)
        r = cor(xvals,yvals)
        a = r*stdy/stdx
        b = meany - a*meanx
        return a,b
    end

    a,b = find_best_fit(xvals,yvals)
    ynew = a * xvals .+ b
    plot!(xvals, ynew)
```

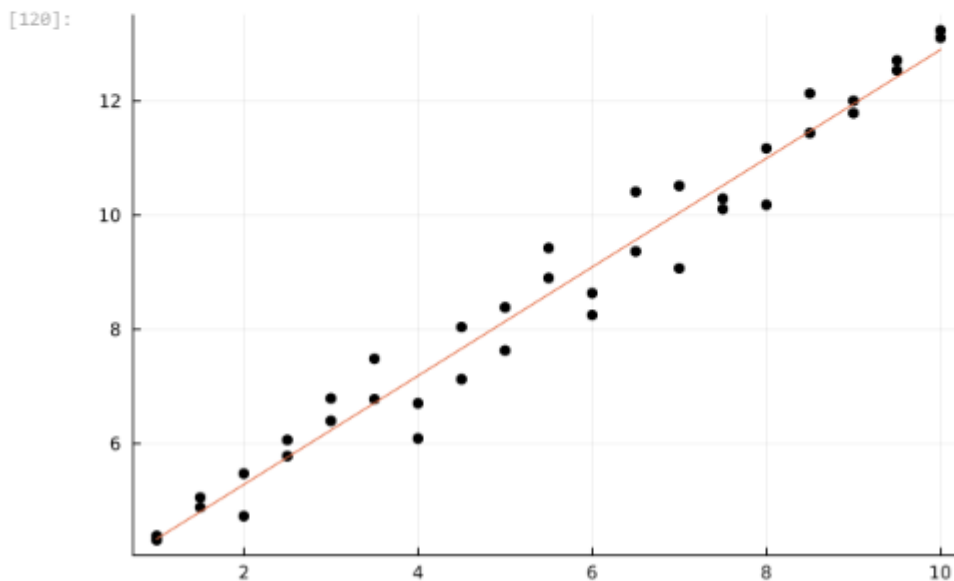


Рис. 35: Применение функции для построения графика

Сгенерируем большой набор данных. Определим, сколько времени потребуется, чтобы найти соответствие этим данным. Для сравнения реализуем подобный код на языке Python. Используем пакет для анализа производительности, чтобы провести сравнение (рис. 36):

```
[122]: xvals = 1:100000;
xvals = repeat(xvals,inner=3);
yvals = 3 .* xvals + 2*rand(length(xvals)) .* 1;
@show size(xvals)
@show size(yvals)
Определим, сколько времени потребуется, чтобы найти соответствие этим данным:
@time a,b = find_best_fit(xvals,yvals)
Для сравнения реализуем подобный код на языке Python:
import Pkg
Pkg.add("PyCall")
Pkg.add("Conda")
using PyCall
using Conda
py"""
import numpy
def find_best_fit_python(xvals,yvals):
    meanx = numpy.mean(xvals)
    meany = numpy.mean(yvals)
    stdx = numpy.std(xvals)
    stdy = numpy.std(yvals)
    r = numpy.corrcoef(xvals,yvals)[0][1]
    a = r*stdy/stdx
    b = meany - a*meanx
    return a,b
"""
xpy = PyObject(xvals)
ypy = PyObject(yvals)
@time a,b = find_best_fit_python(xpy,ypy)
import Pkg
Pkg.add("BenchmarkTools")
using BenchmarkTools
@btime a,b = find_best_fit_python(xvals,yvals)
@btime a,b = find_best_fit(xvals,yvals)
```

Рис. 36: Сравнение

## 2.13 Самостоятельное выполнение

Выполнение задания №1 (рис. 37):

```
[125]: # Загрузка данных
using RDatasets
iris = dataset("datasets", "iris")

# Преобразование DataFrame в матрицу
X = Matrix{Float64}(iris[:, 1:4])

# Количество кластеров
k = 3
result = kmeans(X, k)
# Визуализация кластеров
scatter(X[:, 1], X[:, 2], group = result.assignments, xlabel = "Sepal Length", ylabel = "Sepal Width", title = "K-means Clustering")
```

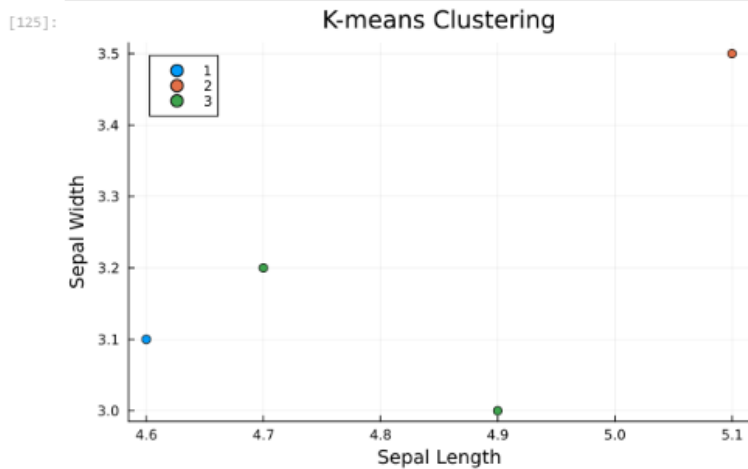


Рис. 37: Решение задания №1

Выполнение задания №2 (рис. 38 - рис. 39):

```
[140]: # Генерация данных
X = randn(1000, 3)
a0 = rand(3)
y = X * a0 + 0.1 * randn(1000)

# Добавляем столбец единиц в X для учета свободного члена
X2 = hcat(ones(1000), X)

# Применяем ридж-регрессию с небольшим значением регуляризации
ridge_result = ridge(X2, y, 1e-4)
println("Ридж-регрессия, коэффициенты: ")
println(ridge_result)

# Сравнение с использованием GLM.jl
# Преобразуем X2 в DataFrame, чтобы использовать его с GLM
df = DataFrame(X2 = hcat(ones(1000), X)..., y = y) # Распаковываем X2 в отдельные столбцы
model = lm(@formula(y ~ X2), df)
println("Результаты с использованием GLM. jl:")
println(coef(model))

Ридж-регрессия, коэффициенты:
[-8.582235320115787e-11, 0.7319523337912158, 0.6359261440048222, 0.10862924800959539, 0.000791333656711028
5]
```

Рис. 38: Решение задания №2

Часть 2.

```
[49]: # Генерация данных
X = rand(100)
y = 2 * X + 0.1 * randn(100)
# Построение графика данных
scatter(X, y, label="Data", xlabel="X", ylabel="y", title="Regression Plot")
# Линейная регрессия
X2 = hcat(ones(100), X) # Добавляем столбец единиц
beta = (X2' * X2) \ (X2' * y)
# Добавление линии регрессии
plot!(X, x -> beta[1] + beta[2] * x, label="Regression Line", color=:red)
```

[49]:

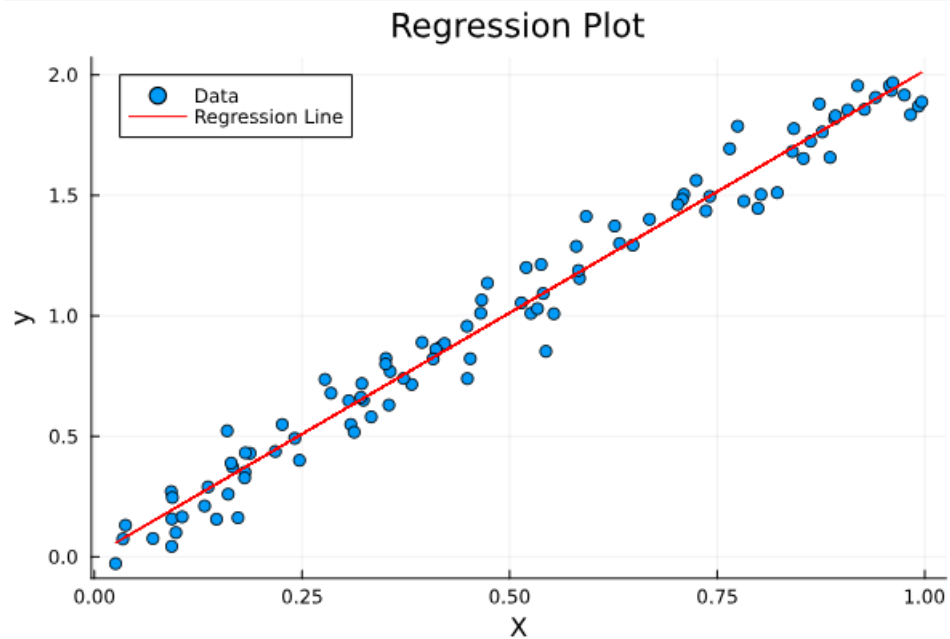


Рис. 39: Решение задания №2

Выполнение задания №3 (рис. 40 - рис. 42):

```
[156]: # Параметры модели
S = 100 # Начальная цена акции
T = 1 # Длительность в годах
n = 10000 # Количество периодов
sigma = 0.3 # Волатильность
r = 0.08 # Годовая процентная ставка
h = T / n # длина одного периода

# Расчет u и d
u = exp(r * h + sigma * sqrt(h))
d = exp(r * h - sigma * sqrt(h))

# Цена акции на каждом шаге
function create_path(S, r, sigma, T, n)
    path = zeros(Float64, n+1)
    path[1] = S
    for i in 2:n+1
        if rand() > 0.5 # Пример случайного выбора направления
            path[i] = path[i-1] * u
        else
            path[i] = path[i-1] * d
        end end
    return path
end

# Генерация и построение траектории
path = create_path(S, r, sigma, T, n)
plot(path, label="Stock Price Path", xlabel="Time Step", ylabel="price", title="Stock Price Trajectory")
```



Рис. 40: Решение задания №3

```
•[159]: # Генерация 10 траекторий
paths = [create_path(S, r, sigma, T, n) for _ in 1:10]

# Построение всех траекторий на одном графике
for path in paths
    plot!(path, label="Trajectory", xlabel="Time Step", ylabel="Price", title="Multiple Stock Price Trajectories")
end
```

Рис. 41: Решение задания №3

```

•[170]: using Threads

# Функция для параллельной генерации траекторий
function parallel_paths(n_paths)
    paths = Vector{Array{Float64, 1}}{undef, n_paths}
    Threads.@threads for i in 1:n_paths
        paths[i] = create_path(S, r, sigma, T, n)
    end
    return paths
end

# Генерация траекторий с использованием многозадачности
paths_parallel = parallel_paths(10)

# Построение всех параллельных траекторий
for path in paths_parallel
    plot!(path, label="Trajectory", xlabel="Time Step", ylabel="Price", title="Parallel Stock Price Trajectories")
end

```

Рис. 42: Решение задания №3

### 3 Вывод

В ходе выполнения лабораторной работы были изучены специализированные пакеты Julia для обработки данных.

### 4 Список литературы. Библиография

[1] Julia Documentation: <https://docs.julialang.org/en/v1/>